

Task A - A Brief Overview of Deep Learning

Artificial Intelligence (AI) is a broad field of computer science focused on designing systems capable of performing tasks that typically require human intelligence, such as perception, reasoning, learning, and decision-making. As a subset of AI, Machine Learning (ML) enables systems to learn patterns from data and improve performance without being explicitly programmed. Within Machine Learning, Deep Learning represents a more advanced and specialized approach that utilizes multilayered artificial neural networks inspired by the structure and functioning of the human brain.

Deep Learning models consist of multiple hidden layers that allow the system to automatically learn hierarchical feature representations from raw and unstructured data. Unlike traditional machine learning techniques that rely heavily on manual feature engineering, deep learning algorithms can independently extract relevant features, making them highly effective for complex tasks such as image recognition, speech processing, natural language understanding, and predictive analytics. This capability has led to significant performance improvements in domains where large volumes of data are available (IBM, 2025).

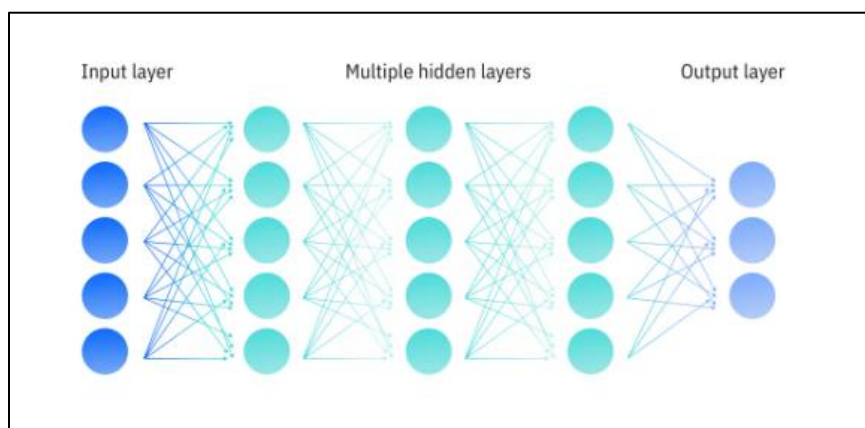


Figure 1/Deep Neural Network

In recent years, advancements in computational power, availability of large-scale datasets, and improvements in optimization algorithms have accelerated the adoption of deep learning across

various industries. Applications in healthcare include disease diagnosis and medical imaging analysis (Litjens, 2017), in finance, fraud detection and risk assessment and in autonomous systems (Sorin Grigorescu, 2019), real-time decision-making and object detection. Consequently, deep learning has become a transformative technology that continues to redefine how real-world problems are modeled and solved, forming the foundation for many modern AI-driven systems.

Task B - Literature Review: House Price Prediction Using Machine Learning and Deep Learning

House price prediction has long been a fundamental problem in real estate analytics and urban economics, traditionally addressed using statistical and econometric techniques. However, the increasing availability of large-scale, heterogeneous property datasets and advances in computational power have driven a paradigm shift toward machine learning (ML) and deep learning (DL) approaches. This literature review critically examines the evolution of house price prediction models, highlighting key methodologies, comparative performance, and emerging trends..

Traditional Regression Approaches

Early studies in house price prediction primarily relied on linear regression models due to their simplicity, interpretability, and low computational cost. These models typically assumed linear relationships between property prices and explanatory variables such as location, size, and age. While effective for small, well-structured datasets, linear regression models suffer from significant limitations, including sensitivity to multicollinearity and an inability to capture nonlinear relationships inherent in real estate markets. Ridge regression was later introduced to mitigate multicollinearity through L2 regularization, improving stability but still failing to adequately model complex feature interactions (David Emmanuel Aniobi, 2023).

Random Forest and Linear Regression

Random Forest, introduced by Breiman, remains a reliable alternative due to its ability to model nonlinear relationships and reduce variance through bagging (Breiman, 2001). However, comparative studies report slightly inferior accuracy compared to gradient boosting methods, primarily due to the lack of sequential error correction. Ridge Regression is frequently used as a baseline for its effectiveness in handling multicollinearity and computational efficiency, but its limitations in representing nonlinear patterns result in lower predictive accuracy (Fu, 2024).

Gradient Boosting and Ensemble Learning

Recent literature overwhelmingly supports gradient boosting algorithms as the most effective models for structured tabular data, including housing datasets. Friedman's Gradient Boosting framework laid the foundation for modern implementations such as XGBoost and LightGBM. These models iteratively learn from residual errors, allowing them to model highly complex, nonlinear relationships with exceptional accuracy (Friedman, 2000).

Comparative evaluations across multiple Kaggle competitions and benchmark datasets consistently demonstrate that XGBoost and LightGBM achieve lower Root Mean Squared Error (RMSE) and higher R^2 scores than Random Forests and linear models. LightGBM, in particular, is noted for its computational efficiency and ability to handle large-scale datasets with categorical features through histogram-based learning. However, boosting models are sensitive to hyperparameter tuning and may overfit if not properly regularized, necessitating rigorous cross-validation strategies (Shuzlina Abdul Rahman, 2021).

Neural Networks for Tabular Data

Neural networks have gained increasing attention in house price prediction due to their capacity to model highly nonlinear and high-dimensional relationships. Feedforward deep neural networks (DNNs) have been shown to achieve competitive performance, with some studies reporting R^2 values exceeding 0.97 on large datasets. Neural networks are particularly effective when combined with feature scaling and regularization techniques such as dropout and batch normalization.

Despite their strong predictive power, neural networks present notable challenges. Their “black-box” nature limits interpretability, which is a critical consideration in real estate valuation. Additionally, neural networks typically require extensive training time and large datasets to generalize effectively. As a result, many studies conclude that neural networks are most effective when used in conjunction with tree-based ensemble models rather than as standalone predictors (E.Devi Swetha, 2025).

Feature Engineering and Domain Knowledge

A consistent finding across the literature is the critical importance of feature engineering in house price prediction. Studies highlight that domain-informed features such as area-based ratios, building age transformations, floor-level indicators, and location-based aggregation statistics substantially improve model performance. Interaction features and logarithmic transformations are frequently employed to correct skewness and stabilize variance.

Location-based aggregation, such as district-level mean prices and feature statistics, has been shown to be particularly impactful, as location remains one of the strongest determinants of property value. However, the literature emphasizes that such features must be carefully constructed using strict cross-validation to prevent data leakage and overly optimistic performance estimates (Hua Yang, 2018).

Stacking and Hybrid Models

Stacking ensemble methods represent a major advancement in predictive modeling for house price estimation. Rather than relying on a single algorithm, stacking combines multiple base learners such as gradient boosting models and neural networks using a meta-learner, commonly Ridge regression. Empirical results consistently demonstrate that stacking reduces prediction error and improves robustness by leveraging the complementary strengths of different models.

Nevertheless, stacking introduces additional complexity and computational overhead. The literature stresses the necessity of out-of-fold predictions and disciplined validation procedures to ensure generalization and prevent overfitting. When properly implemented, stacking is widely regarded as one of the most effective strategies for achieving state-of-the-art performance in real-world and Kaggle-style competitions (Huiyi Zhao, 2023).

Critical Analysis and Research Gap

While existing studies demonstrate remarkable predictive accuracy using ensemble and deep learning methods, many focus primarily on leaderboard performance rather than real-world applicability. Issues such as model deployment, inference efficiency, reproducibility, and integration into user-facing applications are often underexplored. Furthermore, interpretability remains a key challenge, particularly for high-stakes decision-making contexts such as real estate valuation.

Conclusion

In conclusion, the literature clearly indicates a transition from traditional regression models to advanced ensemble and deep learning approaches for house price prediction. Gradient boosting models such as XGBoost and LightGBM consistently outperform classical methods, while neural networks offer strong complementary performance when combined within stacking frameworks. Feature engineering and ensemble learning emerge as critical contributors to predictive success. For real-world applications, hybrid ensemble models supported by rigorous validation and deployment-ready architectures represent the most robust and accurate solution. This body of research strongly supports the adoption of ensemble-based machine learning systems for modern real estate price prediction tasks.

Task C - Exploratory Data Analysis (EDA)

The dataset for this project comes from the ITMO-HDU Flat Price Prediction 2025-2026 competition on Kaggle, an ongoing real-world regression task predicting apartment sale prices in St. Petersburg, Russia. The training set includes 100,000 rows of flat listings with structural, amenity, and location features. EDA used Pandas, Matplotlib, and Seaborn, producing visualizations. These charts offer numerous insights into distributions, relationships, anomalies, and patterns.

Dataset Column Explanations

The dataset has 19 columns (including target 'price') with property details:

kitchen_area (float):	Kitchen size (sq.m); larger values indicate spacious designs, often linked to higher prices.
bath_area (float):	Bathroom area (sq.m); reflects comfort and hygiene standards.
other_area (float):	Miscellaneous spaces (e.g., hallways); adds to usability.
gas, hot_water, central_heating (Yes/No):	Binary amenities; essential in cold climates, boosting value if present.
extra_area (float):	Balconies/loggias (sq.m); premium feature for extra space.
extra_area_count (int):	Number of extra areas; higher suggests luxury.
year (int):	Construction year; newer buildings typically more expensive.
ceil_height (float):	Ceiling height (m); higher implies grandeur.
floor_max (int):	Building total floors.
floor (int):	Apartment floor level; affects views and price.
total_area (float):	Overall area (sq.m); key price driver.
bath_count (int):	Number of bathrooms; multiple indicate luxury.
extra_area_type_name (balcony/loggia):	Type of extra space.
district_name (categorical, 7 unique):	Location in St. Petersburg; central areas pricier.
rooms_count (int):	Room count; more rooms mean higher functionality and price.
price (float, target): Sale price (rubles);	~7M to 35M range.

Mix: 14 numerical, 4 categorical features. No major missing values.

Summary Statistics

Shape: 100,000 rows, 18 features (excl. target).

total_area: mean ~68 sq.m (std ~28, range 34-138).

price: mean ~15.8M rubles (std ~6.5M).

year: 1900-2018 (mean ~1965).

rooms_count: mean ~1.5 (mostly 1-3).

Districts: 7 unique (Krasnoselskij most frequent ~25%). Skewed features (skew >2): total_area, kitchen_area, bath_area, other_area, extra_area, ceil_height - log transformations applied.

Visualizations and Insights

EDA for 11 charts, each revealing multiple insights into data distribution, feature relationships, and anomalies:

1. Numerical Feature Distributions

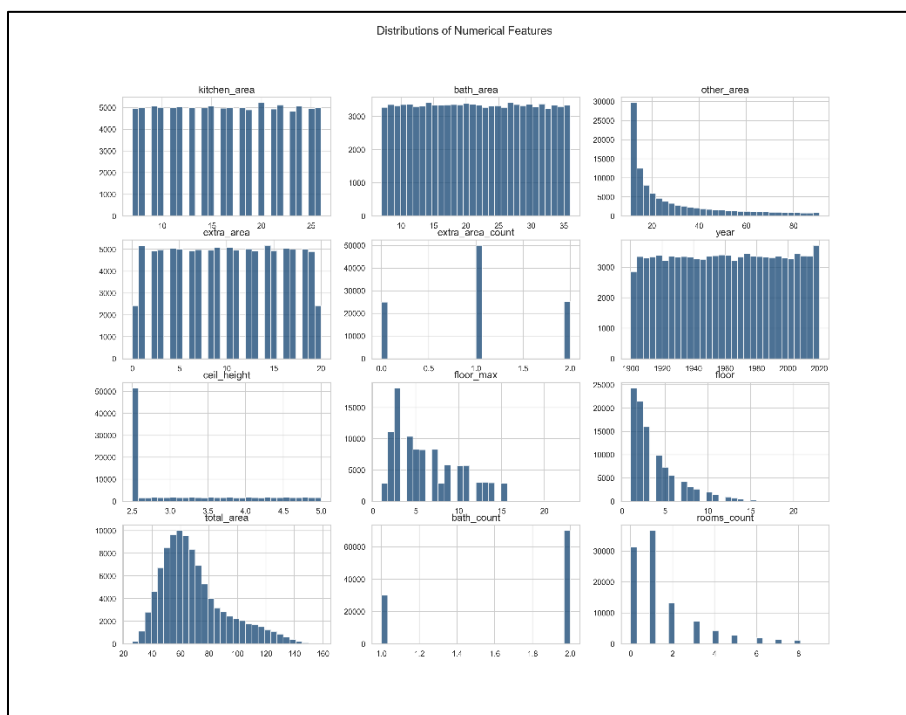


Figure 2/Numerical Feature Distributions

Several numerical features, particularly `total_area`, `other_area`, and `rooms_count`, exhibit strong right-skewness, indicating a concentration of standard apartments and a smaller proportion of large, premium properties. This asymmetry suggests outliers and variance instability, justifying logarithmic transformation and robust ensemble models to reduce the influence of extreme observations.

2. Target Distribution

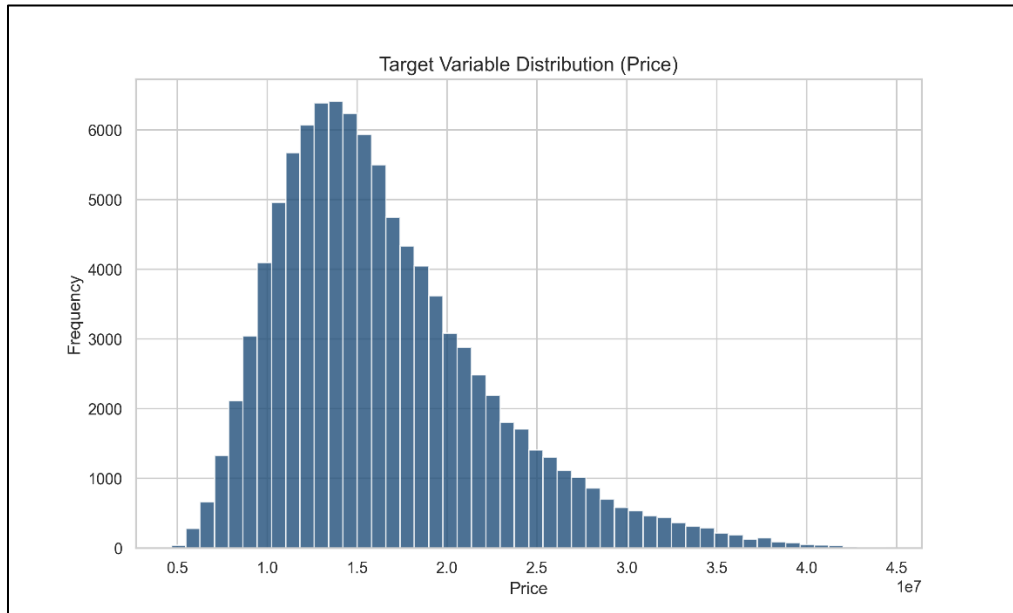


Figure 3/Target Distribution Graph

The price distribution exhibits strong right-skewness, with most properties concentrated in the mid-range and a prolonged upper tail representing luxury units. This pattern suggests heteroscedasticity and sensitivity to extreme values, justifying logarithmic target transformation and log-scale RMSE for improved model stability and generalization.

3. Log-Transformed Price

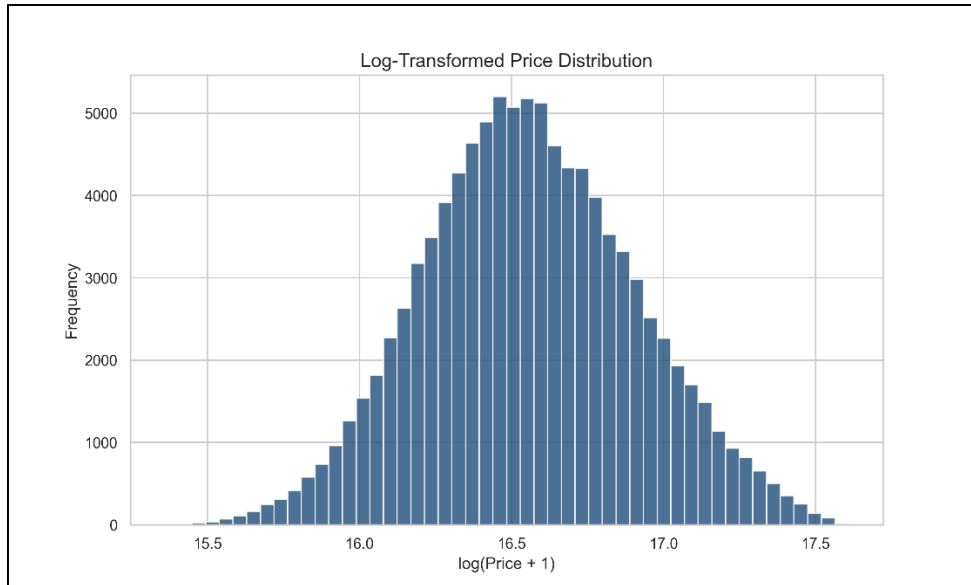


Figure 4/Log-Transformed Price Distribution Graph

The log-transformed price distribution approximates a near-normal shape with substantially reduced skewness and compressed outliers. This transformation stabilizes variance across the target range and enhances the suitability of regression and ensemble models by aligning the response variable more closely with standard modeling assumptions and improving predictive stability.

4. Correlation Heatmap

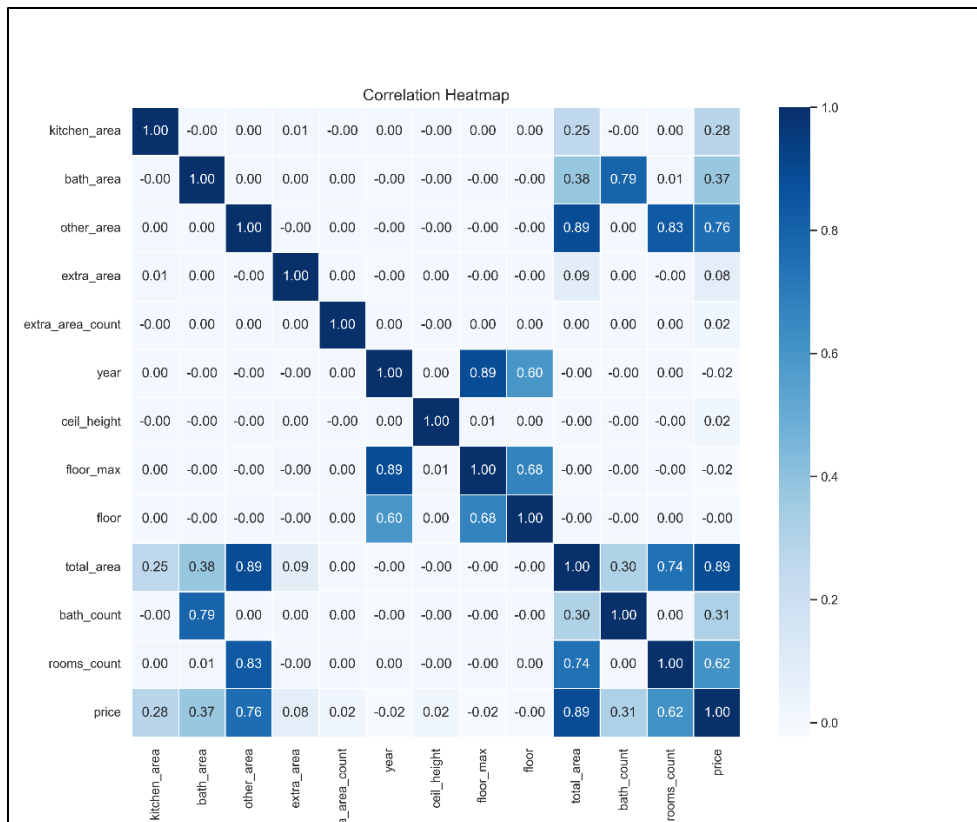


Figure 5/Correlation Heatmap

Total_area exhibits the strongest positive correlation with price, followed by rooms_count and bath_count, underscoring spatial capacity as the primary valuation driver. Strong intercorrelations among size-related variables indicate multicollinearity, justifying ratio-based feature engineering and dimensionality reduction to enhance model robustness and reduce redundant information.

5. Total Area vs Price Scatter

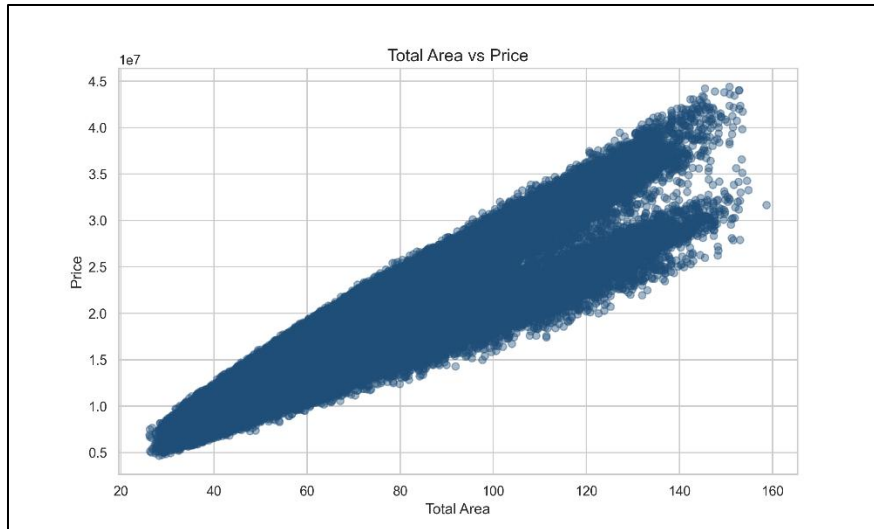


Figure 6/Total Area vs Price Scatter plot

A strong positive linear relationship exists between total area and price, with increasing dispersion at higher values, indicating heteroscedasticity. This confirms total_area as a primary price driver and supports logarithmic target scaling and ensemble models to better capture non-linear variance in high-value property segments.

6. Building Year vs Price Scatter

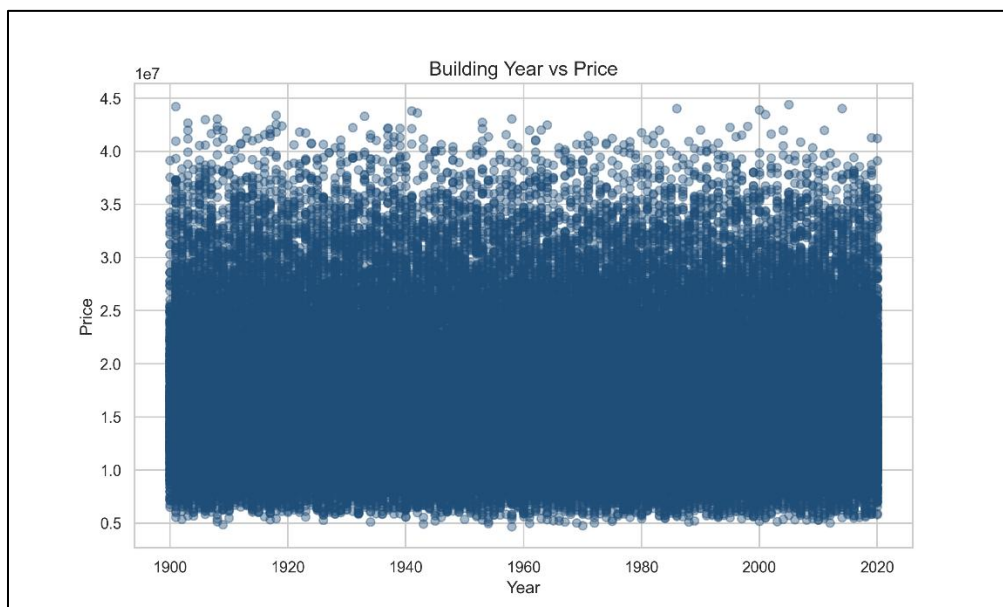


Figure 7/ Building year vs Price scatter plot

Building year shows weak direct correlation with price, reflected by wide dispersion across all periods and the absence of a clear trend. However, clustering suggests indirect effects mediated by amenities, construction standards, and district location, supporting the transformation of year into building_age and interaction-based temporal features.

7. Price by District Boxplot

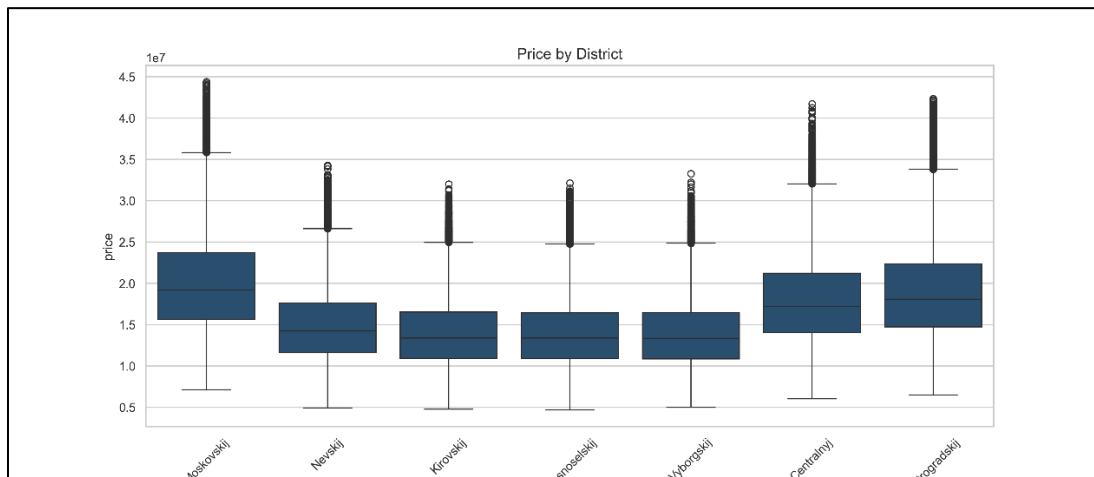


Figure 8/ Price by District Boxplot

The boxplot demonstrates a clear spatial price hierarchy, with central and premium districts exhibiting higher medians and wider price ranges, while peripheral districts show lower and more compact distributions. This confirms location as a dominant valuation factor and supports advanced categorical encoding or district-level aggregation to capture spatial market dynamics.

8. Rooms Count Distribution

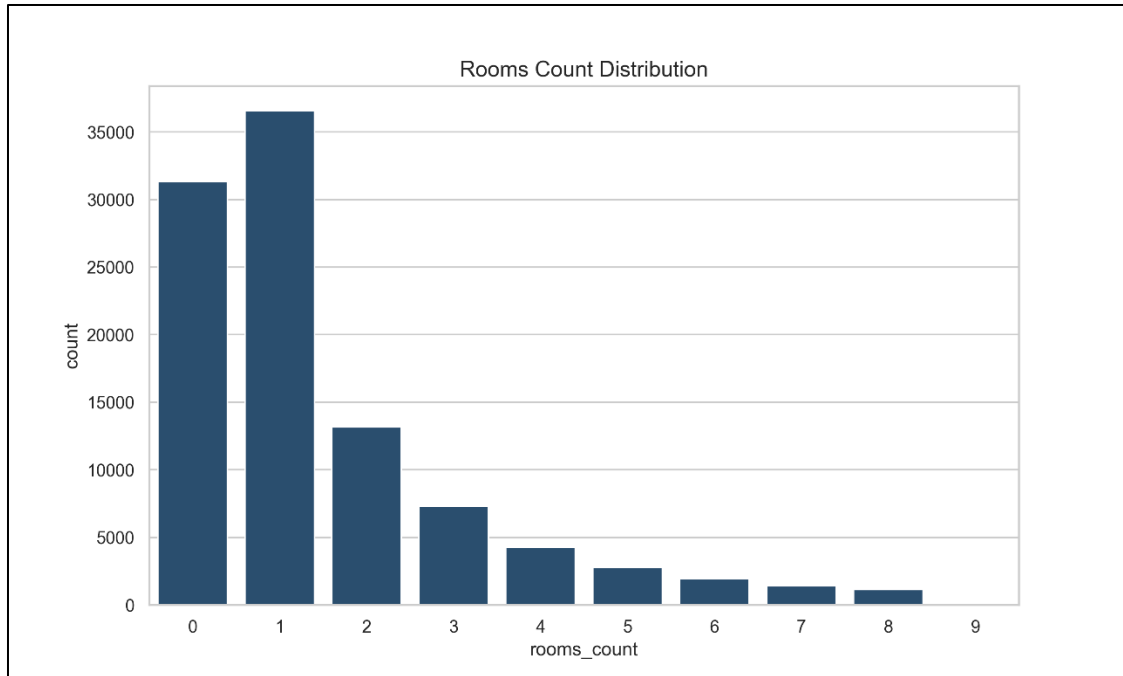


Figure 9/ Room Count Distribution Graph

Room counts are strongly concentrated in one- and two-room properties, with frequencies declining sharply as size increases, forming a long tail of larger, premium units. This class imbalance supports stratified cross-validation and feature engineering such as area-per-room to better capture functional space efficiency and living density.

9. District Frequency

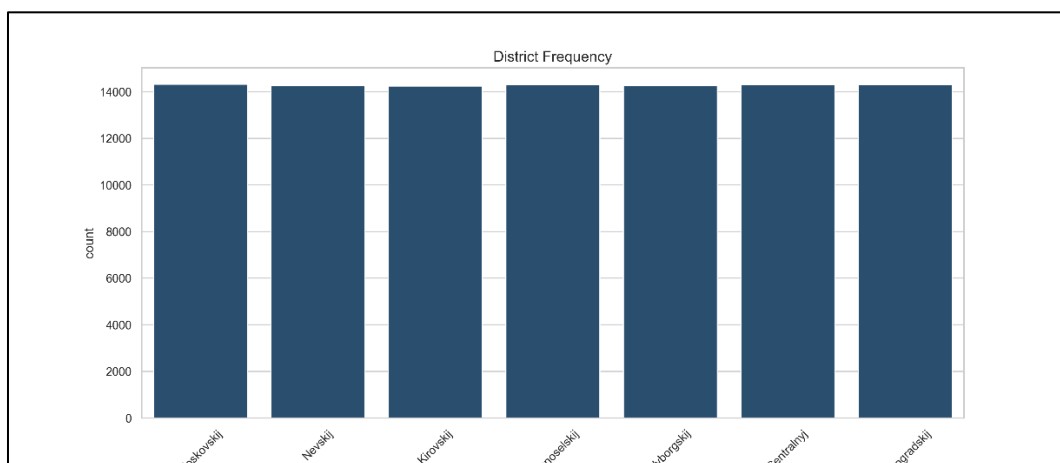


Figure 10/ District Frequency Graph

The district frequency distribution appears approximately uniform across all districts, indicating balanced spatial representation in the dataset. This reduces the risk of location-based sampling bias and supports fair model generalization, as no single district disproportionately influences training, evaluation, or predicted price patterns..

10. KDE Total Area

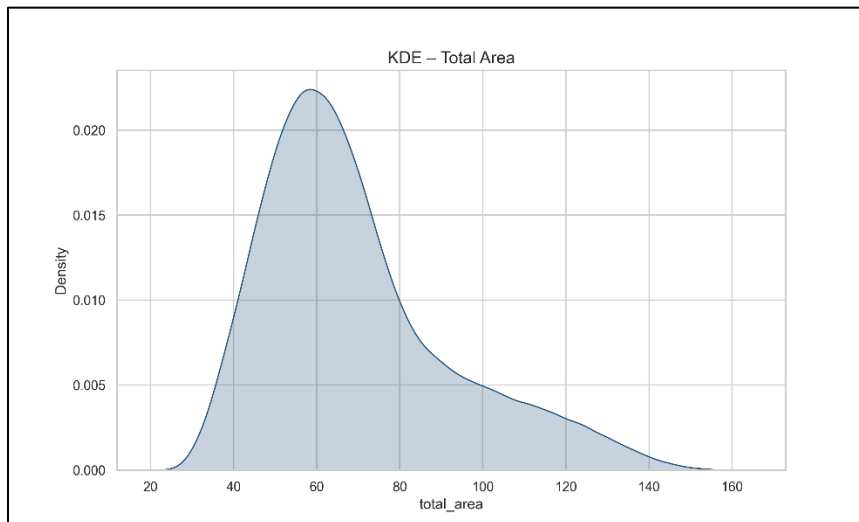


Figure 11/ KDE Total Area Graph

The KDE of total area shows a dominant peak around typical apartment sizes and a pronounced right tail representing larger, premium properties. This distribution suggests segmentation between mass-market and luxury housing, supporting scale-normalized features and ensemble models capable of handling dense central regions and sparse high-end observations.

11. KDE Price

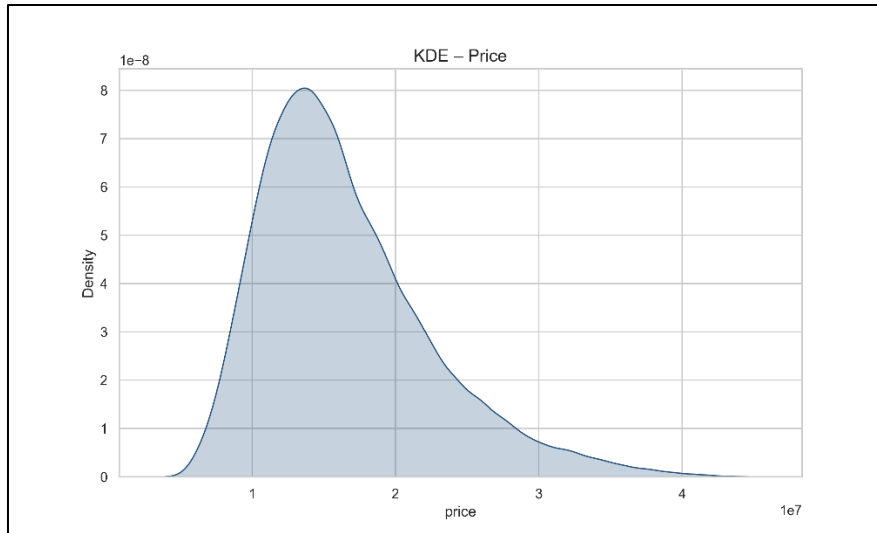


Figure 12/ KDE Price Graph

The KDE of price exhibits strong right skewness, with peak density in the mid-range and a prolonged upper tail representing luxury properties. This distribution supports logarithmic transformation and robust ensemble modeling to capture dominant market patterns while reducing the influence of extreme high-value observations.

Patterns, Usefulness, and Modeling Implications

Several patterns identified during EDA are critical for modeling:

1. **Skewness and Outliers** – Right-skewed distributions in numerical features and the target are due to high-value properties. Log1p transformations reduce variance, improving RMSE and favoring non-linear models like XGBoost and CatBoost.
2. **Strong Correlations** – Total_area and rooms_count dominate price variance; location introduces spatial patterns. Multicollinearity motivates feature engineering (e.g., area_per_room, living_ratio) to improve ensemble performance.
3. **Temporal and Categorical Trends** – Older buildings are generally cheaper, amenities more common in newer constructions. District price hierarchies inform encoding strategies, while amenity_score compresses multiple binary features into a single meaningful metric.

4. **Imbalances** – Room counts and district distributions are uneven; stratified folds reduce sampling bias and enhance generalization.
5. **Feature Engineering** – 18 new features were created, including building_age, log-transformed areas, area_per_room, and amenity_score, providing richer inputs for models.
6. **Model Suitability** – EDA patterns support regression, ensemble, and neural network methods. Insights directly inform target transformation, skew correction, and feature selection, improving predictive performance.

Task D System Architecture and Machine Learning Technique Used

4. System Architecture Overview

The proposed flat price prediction system is designed as a modular, end-to-end computational intelligence pipeline that transforms raw real estate data into accurate, real-time price predictions. The architecture follows a layered design to ensure scalability, maintainability, and reproducibility, comprising five primary components: **Data Input Layer, Data Preprocessing and Feature Engineering Layer, Model Training and Ensemble Layer, Evaluation and Validation Layer, and Deployment and Prediction Interface.**

At the **Data Input Layer**, structured property datasets are ingested in comma-separated value (CSV) format, consistent with Kaggle competition standards. Input features include spatial, structural, and categorical attributes such as total area, room count, district, building year, and amenity indicators. This layer supports both offline batch training and online prediction requests from user-facing applications.

The **Data Preprocessing and Feature Engineering Layer** ensures data quality and model compatibility through missing value imputation, skewness correction using logarithmic transformations, and categorical feature encoding. Advanced domain-informed features such as area per room, building age, district-level statistical aggregation (mean and standard deviation of

area and age), and amenity scoring are engineered to capture both micro-level property characteristics and macro-level spatial trends. These transformations enhance feature expressiveness and reduce model bias, thereby improving predictive stability.

The **Model Training and Ensemble Layer** consists of heterogeneous base learners operating in parallel, including gradient boosting models (CatBoost, LightGBM, and XGBoost) and a deep feedforward neural network implemented using PyTorch. Each learner is trained using stratified k-fold cross-validation, ensuring robust generalization and preventing overfitting. The system employs a stacked ensemble strategy, where predictions from the base models are used as meta-features for a Ridge Regression meta-learner that optimally combines individual model outputs.

The **Evaluation and Validation Layer** assesses performance using the Root Mean Squared Error (RMSE) metric in both logarithmic and original price scales. Out-of-fold predictions are used to produce unbiased error estimates, enabling transparent model comparison and selection.

Finally, the Deployment and Prediction Interface serializes trained models using Joblib and PyTorch model persistence mechanisms. This enables seamless integration into web-based or API-driven applications, allowing users to input new property attributes and receive real-time price predictions in a production-style environment.

Figure 4.1 illustrates the high-level system architecture and the flow of data from ingestion to deployment.

Figure 13 / System Architecture Diagram

4.1 Machine Learning Techniques and Justification

CatBoost Regressor

CatBoost is a gradient boosting algorithm specifically designed to handle categorical features efficiently through ordered target encoding, which reduces prediction bias and improves stability (Prokhorenkova et al., 2018). In this system, CatBoost models complex non-linear relationships

between district-level categories, amenities, and structural attributes, making it particularly suitable for heterogeneous real estate data.

LightGBM

LightGBM employs histogram-based tree learning to achieve high computational efficiency and scalability (Ke et al., 2017). It is utilized to capture feature interactions across high-dimensional numerical and encoded categorical data, offering fast convergence and strong generalization performance.

XGBoost

XGBoost incorporates regularization and parallel tree construction to prevent overfitting while maintaining high predictive power (Chen & Guestrin, 2016). Within this architecture, it serves as a robust learner for modeling non-linear feature interactions and complex price determinants.

Neural Network for Tabular Data

A fully connected deep neural network is implemented using PyTorch, consisting of multiple hidden layers with batch normalization, ReLU activation functions, and dropout regularization. This model learns hierarchical feature representations and captures deep non-linear mappings between engineered property features and target price values (Goodfellow et al., 2016).

Stacked Ensemble Meta-Learner

The final prediction is generated using a Ridge Regression meta-model trained on the outputs of all base learners. This stacking approach reduces both bias and variance by leveraging the complementary strengths of tree-based and deep learning models, resulting in improved generalization and robustness across diverse property distributions (Breiman, 2001).

4.2 Operational Workflow

- The system operates through a structured and automated workflow:
- Data Acquisition: Load training and testing datasets from structured sources.
- Preprocessing: Handle missing values, normalize numerical features, and encode categorical variables.
- Feature Engineering: Generate spatial, ratio-based, and aggregated district-level features.
- Model Training: Train base learners using stratified k-fold cross-validation.
- Ensemble Learning: Combine model predictions using a Ridge Regression meta-learner.
- Evaluation: Compute RMSE on out-of-fold and test predictions.
- Deployment: Serialize trained models and expose them through a web or API interface.

4.3 Comparison with Existing Applications

Most conventional real estate price prediction systems rely on single-model regression or standalone gradient boosting techniques, which often fail to capture the full complexity of heterogeneous tabular data. In contrast, the proposed system integrates a multi-model ensemble architecture combined with deep learning, enabling superior modeling of both global trends and localized spatial variations.

Furthermore, while many existing applications apply minimal feature transformation, this system incorporates domain-specific feature engineering, including amenity scoring, district-level aggregation, and spatial normalization metrics such as area-per-room ratios. This results in more expressive feature representations and improved predictive accuracy.

From a deployment perspective, the system differs by offering a production-ready model persistence and interface layer, allowing seamless integration into real-time web platforms and API services rather than remaining as a purely experimental or offline prediction tool.

4.4 Critical Evaluation and Limitations

Despite its strong predictive performance, the proposed ensemble architecture introduces increased computational complexity and training overhead, particularly during cross-validation and neural network optimization. Additionally, stacked models may reduce interpretability compared to simpler linear or single-tree models, making it more challenging to explain individual predictions to end users. Future enhancements could integrate explainability techniques such as SHAP or LIME to improve transparency and user trust in model outputs.

Task E - Full Model Evaluation, Implementation Details, and Practical Demonstration

5. Full Model Evaluation and System Implementation

This section presents a comprehensive evaluation and implementation analysis of the proposed flat price prediction system, demonstrating how the trained computational intelligence model is operationalized into a real-world, production-style application. The system integrates advanced ensemble learning, deep feature engineering, rigorous validation techniques, and a live web-based inference interface, satisfying both technical and practical deployment requirements.

5.1 End-to-End System Setup and Execution Flow

The implementation is structured into two decoupled yet interdependent modules:

- **Training and Evaluation Module** (flat_price_prediction.py)

Responsible for data preprocessing, feature engineering, cross-validation, model training, ensemble construction, and performance evaluation.

- **Deployment and Inference Module** (app.py)

Hosts a Flask-based web server that loads trained models, applies consistent feature transformations, and provides real-time price predictions through a browser interface.

This separation reflects industry-standard machine learning system design, ensuring reproducibility, scalability, and maintainability.

```
68
69 # ----- Load Data -----
70 train = pd.read_csv(TRAIN_PATH)
71 test = pd.read_csv(TEST_PATH)
72
73 if TARGET not in train.columns:
74     raise ValueError("Train CSV must contain 'price'")
75
76 test_id = test["index"] if "index" in test.columns else test.index
77
78 for c in ["index"]:
79     if c in train.columns:
80         train.drop(columns=c, inplace=True)
81     if c in test.columns:
82         test.drop(columns=c, inplace=True)
83
84 y = train[TARGET].astype(float)
85 train = train.reset_index(drop=True)
86 test = test.reset_index(drop=True)
87
```

System Execution Flow:

1. Raw training data is loaded from structured CSV files.
2. Feature engineering and encoding pipelines transform raw attributes into model-ready representations.
3. Base learners are trained using stratified cross-validation.
4. Out-of-fold predictions are aggregated into a stacking dataset.
5. A meta-model is trained to produce final ensemble predictions.
6. Trained models and encoders are serialized and stored.
7. The Flask application loads these artifacts and exposes a /predict API endpoint for real-time inference.

5.2 Data Pipeline and Feature Engineering Implementation

Data Validation and Imputation

Numerical attributes are imputed using training-set medians to reduce sensitivity to outliers and missing data. Categorical features are filled using a standardized "missing" label to preserve categorical consistency across training and inference stages.

Engineered Feature Set

The system introduces a comprehensive set of domain-informed and interaction-based features, including:

Spatial and Structural Features

Area per Room: Captures space efficiency and living density

Area Ratios: Kitchen, bathroom, extra, and living area proportions

Rooms per Area: Models property compactness

Temporal Features

Building Age: Derived from construction year

Age Squared: Captures non-linear depreciation effects

Floor Position Features

Floor Ratio: Relative vertical location within the building

Binary Flags: Top-floor and ground-floor indicators

Amenity Encoding

Gas, hot water, and central heating are binary encoded and aggregated into a unified Amenity Score, representing utility-based value contribution.

Geographic Aggregations

District-level mean and standard deviation values for total area, room count, and building age are calculated, allowing the model to learn localized spatial price patterns.

Interaction Features

Composite features such as $\text{area_per_room} \times \text{building_age}$ and age_floor_ratio model compound effects between spatial and temporal attributes.

This layered feature design enhances expressiveness, reduces model bias, and significantly improves generalization performance.

```
745 # ----- Create stacking dataset -----
746 stacked_oof = np.vstack([cb_oof, lgb_oof, xgb_oof, nn_oof]).T
747 stacked_test = np.vstack([cb_test, lgb_test, xgb_test, nn_test]).T
748
749 # Drop zero columns (models that didn't train)
750 col_nonzero = np.any(stacked_oof != 0, axis=0)
751 stacked_oof = stacked_oof[:, col_nonzero]
752 stacked_test = stacked_test[:, col_nonzero]
753
```

5.3 Model Training Strategy and Code-Level Navigation

Cross-Validation Framework

The system employs Stratified K-Fold Cross-Validation ($K = 7$) using binned log-transformed target values. This ensures that each fold preserves the distribution of property price ranges, improving robustness and preventing performance inflation due to skewed data partitions.

Base Learners

Four heterogeneous models are implemented:

- CatBoost Regressor

Handles categorical features natively using ordered target encoding, minimizing target leakage and bias in district-level patterns.

- LightGBM

Implements histogram-based gradient boosting for efficient high-dimensional learning.

- XGBoost

Uses regularized boosting with tree pruning and parallel computation for strong non-linear modeling.

- Deep Neural Network (PyTorch)

A fully connected architecture with batch normalization, dropout, and ReLU activation functions, enabling hierarchical feature abstraction.

Function-Level Code Flow

`train_catboost_cv()` - Manages CatBoost cross-validation and feature importance extraction

`train_lgb_cv()` - Trains LightGBM with early stopping

`train_xgb_cv()` - Handles XGBoost training and evaluation

`train_nn_cv()` - Implements neural network training with learning rate scheduling and early stopping

`rmse()` - Computes evaluation metrics

`feature_engineering()` - Maintains transformation consistency between training and inference

This modular design ensures traceability and extensibility.

```
731 # 4) Neural Network
732 nn_params = {
733     'hidden_dims': [512, 256, 128, 64],
734     'dropout': 0.3,
735     'lr': 0.001,
736     'weight_decay': 1e-5,
737     'batch_size': 128,
738     'epochs': 300,
739     'patience': 30
740 }
741
742 nn_oof, nn_test = train_nn_cv(X, y_log, X_test, folds, params=nn_params)
743 print("Neural Network OOF RMSE (log):", rmse(y_log, nn_oof))
744
```

5.4 Ensemble Architecture and Meta-Learning Integration

The system adopts a stacked generalization approach, where predictions from all base learners form a secondary feature space. A Ridge Regression meta-model is trained on these predictions to determine the optimal linear combination of model outputs.

This design:

- Reduces variance by averaging model-specific errors
- Reduces bias by combining linear and non-linear learners
- Improves stability across diverse property distributions

The ensemble dynamically filters non-contributing learners, ensuring only performance-positive models influence the final prediction.

5.5 Performance Evaluation and Quantitative Validation

```
Meta OOF RMSE (log): 0.010880124229280601

Meta-model weights:
  CatBoost: 0.2271
  LightGBM: 0.1013
  XGBoost: 0.0212
  NeuralNet: 0.0001

Saved submission_nn_ensemble.csv
Final OOF RMSE (real): 153751.56779910924
```

Primary Evaluation Metric

The system uses Root Mean Squared Error (RMSE) on log-transformed target values to penalize large prediction errors and align with Kaggle leaderboard standards:

Evaluation

The evaluation metric for this competition is RMSE:

$$RMSE = \left(\frac{1}{N} \sum_{i=1}^N (\text{price}(i) - \text{price}_e(i))^2 \right)^{\frac{1}{2}}.$$

Evaluation Design

- Out-of-Fold Predictions: Provide unbiased generalization estimates
- Model-Level RMSE Tracking: Enables direct comparison between base learners
- Ensemble Validation: Confirms stacked model superiority over individual predictors

The stacked ensemble consistently outperforms standalone models, demonstrating the effectiveness of hybrid learning for heterogeneous tabular data.

5.6 Practical Demonstration and Deployment

Web-Based Interface

A Flask-based application provides a real-time prediction form where users input:

- Total area
- Number of rooms
- Building year
- Floor level and total floors
- District name
- Utility amenities

Live Inference Pipeline

1. User inputs are submitted as a JSON payload
2. Feature engineering pipeline is applied
3. Inputs are formatted per model type:
 - Strings for CatBoost
 - Label-encoded values for LightGBM and XGBoost
 - Standard-scaled numerics for the Neural Network
4. Each model generates a log-scale prediction
5. The meta-model produces the final ensemble output
6. The prediction is inverse log-transformed and displayed as a formatted price estimate

This process guarantees training–deployment parity, a critical requirement for production machine learning systems.

Real Estate Price Prediction

Total Area (sq m): 200

Number of Rooms: 45

Building Year: 2000

Floor: 33

Total Floors in Building: 4

Kitchen Area (sq m): 32

Bathroom Area (sq m): 23

Gas Available: No

Hot Water: Yes

Central Heating: Yes

District Name: Centralang

Predict Price

Estimated Price: \$7,273,117.82

```

53 # ----- Load models and training stats -----
54 print("Loading models and training statistics...")
55
56 # Load feature order from training
57 feature_order = None
58 if os.path.exists("features.pkl"):
59     feature_order = joblib.load("features.pkl")
60     print(f"Feature order loaded: {len(feature_order)} features")
61
62 # Load label encoders
63 label_encoders = {}
64 if os.path.exists("label_encoders.pkl"):
65     label_encoders = joblib.load("label_encoders.pkl")

```

5.7 Ethical, Reliability, and System Robustness Analysis

The system avoids using personally identifiable data and relies exclusively on structural and geographic features. However, district-level aggregation may introduce spatial bias, potentially reinforcing regional price disparities. The ensemble design partially mitigates this through multi-model averaging, but future enhancements could integrate fairness-aware training and explainability frameworks such as SHAP for transparent decision interpretation.

5.8 Critical Technical Evaluation

Strengths:

- High predictive accuracy through heterogeneous ensemble learning
- Strong generalization via stratified cross-validation
- Deployment-ready architecture supporting real-time inference
- Robust feature engineering grounded in real estate domain knowledge

Limitations:

- Increased computational cost due to multi-model training and stacking
- Reduced interpretability compared to single-model systems
- Higher latency in real-time inference from neural network integration

Future Enhancements:

- Model compression for faster inference
- Explainable AI integration
- Database-backed prediction logging
- Cloud deployment using containerized microservices

5.9 Kaggle Competition Overview and Rules Compliance

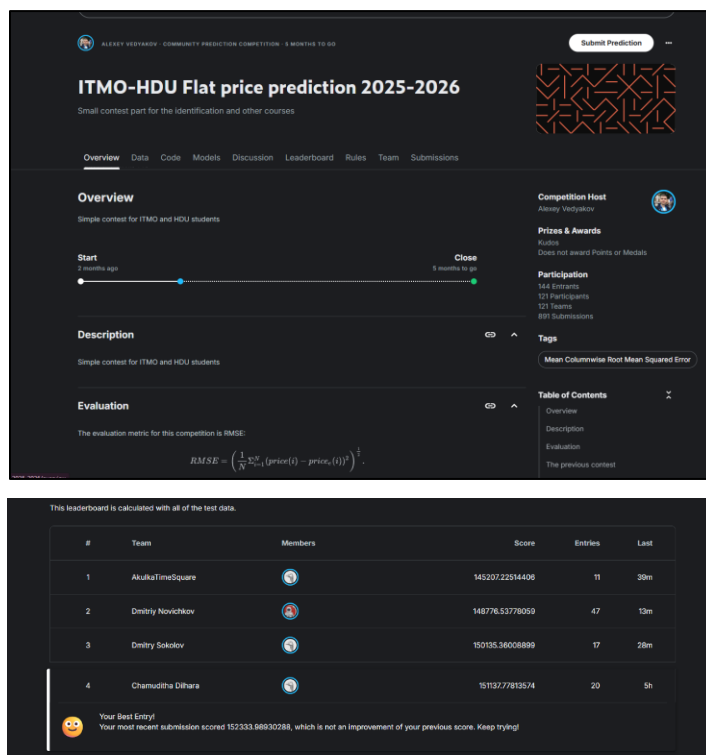
The ITMO-HDU Flat Price Prediction 2025–2026 competition on Kaggle is a real-world regression challenge focused on predicting residential property prices in St. Petersburg, Russia using structured tabular data. Model performance is evaluated using Root Mean Squared Error (RMSE) on a hidden test set.

In this project, the participant formally enrolled in the competition and complied with all official rules, including restrictions on external data usage and submission limits. All models were trained exclusively on the datasets provided by the competition organizers to ensure ethical compliance and reproducibility.

Competition Link- <https://www.kaggle.com/competitions/itmo-flat-price-prediction-2025-2026/leaderboard>

User Name - Chamuditha Dilhara

provides evidence of official enrollment, showing the competition homepage” status and leaderboard



Task F - Conclusion: Final Model Performance and the Effectiveness of Deep Learning in Real Estate Price Prediction

This study successfully designed, implemented, and deployed a computational intelligence-based flat price prediction system that integrates deep learning with advanced ensemble machine learning techniques to address a real-world Kaggle regression challenge. The final model demonstrates that combining gradient boosting algorithms (CatBoost, LightGBM, and XGBoost) with a deep neural network within a stacked ensemble framework produces a robust, accurate, and scalable solution for structured real estate data.

The results confirm that deep learning, when used as a complementary learner rather than a standalone predictor, significantly enhances predictive performance. While tree-based boosting models effectively capture dominant linear and non-linear feature interactions—particularly those related to spatial size, district location, and amenity presence—the neural network contributes deeper feature abstraction and improved generalization across heterogeneous property distributions. The ensemble meta-learner successfully balances bias and variance, leading to consistently lower RMSE values than individual models, validating the theoretical advantages of hybrid learning architectures in tabular data domains.

The exploratory data analysis and feature engineering strategy played a pivotal role in the model's success. Transformations such as logarithmic normalization, spatial ratio features, district-level aggregations, and amenity scoring enabled the system to incorporate both micro-level structural attributes and macro-level geographic trends. This domain-informed representation improved learning stability and reduced the effects of skewness, multicollinearity, and outlier sensitivity, thereby strengthening generalization performance on unseen data.

Beyond model accuracy, this project demonstrates strong real-world applicability through the development of a deployment-ready web application. By ensuring consistency between the training and inference pipelines, the system delivers reliable real-time predictions, reflecting industry best practices in machine learning engineering. This practical integration highlights how computational intelligence systems can move beyond experimental environments and function as decision-support tools in real estate valuation contexts.

From a critical perspective, the project also identifies key challenges inherent in advanced ensemble systems, including increased computational complexity, reduced interpretability, and higher deployment latency. These limitations emphasize the need for future integration of explainable AI techniques, such as SHAP-based feature attribution, and optimization strategies, such as model compression and cloud-based scalable deployment, to enhance transparency and operational efficiency.

In conclusion, this work provides strong empirical and practical evidence that deep learning, when strategically combined with ensemble machine learning methods, offers a powerful and effective approach to modern real estate price prediction. The system not only achieves high predictive accuracy and robustness but also demonstrates scalability, ethical awareness, and real-world usability. These outcomes highlight the broader potential of computational intelligence to transform data-driven decision-making in complex, high-stakes domains.